

Report on Optimal Controller Design for a Mobile Robot Using Genetic Algorithm and Adaptive PID Controller

Rishi Mehta and Rahul Ravi Vk

University of Maryland College Park

November 20, 2025

Abstract

With this report, we tried to recreate a solution to the problem of steering a robot to a specific position with a desired orientation using optimal kinematic control in the lowest time possible. Also, when the dynamics of a model are introduced, it becomes impossible to compute optimal control gains analytically. This study proposes a solution to separate low-level controls from aspects of planning. First, at the kinematic level, we used an optimal trajectory generator using cubic polynomial interpolation and tuning its parameters using a genetic algorithm, which produced a smooth, time-efficient path to the goal. At the dynamic level, an adaptive PID controller is used to balance out the unmodeled dynamics and varying parameters of the robot, such as mass, to ensure a stable and robust tracking of the plan. The results proved that with adaptive PID, the performance is consistent under changes of mass and external disturbances. Across different scenarios, this approach is consistent with smaller time errors. The end result is an improved practical mobile robot controller design with future work related to tighter integration of both kinematic and dynamic controllers to achieve fully autonomous operations to reach arbitrary poses.

Keywords Kinematic controller, Optimal control, Dynamic model, Cubic interpolation, Genetic algorithm, Adaptive PID, Gradient Descent Method.

1 Introduction

In the modern industrial domain, mobile robot technology is at the forefront of maximizing automation and efficiency, with particular applications in the logistics, manufacturing, and service industries. These are the sectors where precise trajectory planning and control are key factors to improve efficiency and productivity. Despite such advances in this technology, it remains a challenge due to the complex dynamics

and various environmental factors [1], [2], [3]. The control system of such mobile robots directly affects their performance, operational range, mobility and accuracy. The mobile robot control can be broadly divided into two areas: first, generating a path for the robot to follow, and second is the optimal control to minimize the path-tracking error while following the generated trajectory [4], [5]. The traditional approach of controlling a mobile robot relies on adjusting the wheel speeds to reach a particular position and rotation based on the kinematic controllers output, while disregarding the dynamic elements of the robot such as acceleration and torque, which is suitable for low-speed operations where inertia can be ignored [6], [7], [8]. Also, there are limitations to this type of control regarding reaching the target in a minimal time. Kinematic controllers often produce control outputs exceeding the physical limits of the system, such as the maximum speed and angular velocity for the robot.

When viewing the problem of moving a mobile robot to a desired location, it appears to be a complex non-linear optimal control problem, which can be solved by involving dynamics along with traditional kinematic methods [9]. Various techniques have been explored that utilize the robot dynamics to enhance the real-time control performance of the system, such as using neural network-based controllers proposed by Fierro and Lewis [10]. A modular approach was implemented by Nandy et al. [11] to enhance the accuracy and efficiency in real-time gain scheduling. Martin and Brandão [12] proposed a dynamic compensation control system by applying maximum speed limits to improve safety and reduce path tracking error. The main focus of these approaches is to improve the path tracking performance, and they often miss out on the important aspect of optimal trajectory generation. While Choi and Park [9] optimized the linear controller gains using genetic algorithms and neural networks to make robots reach target pose quickly, but they missed out the robot's dynamic parameters, leading to potential issues regarding stability in the control system. Recently, Park et al. [4] addressed stability issue by setting kinematic controller gains while considering the dynamic constraints such as maximum speed and angular velocity.

Genetic Algorithms, which mimic real-life biological evolution processes, are a well-known machine learning optimization algorithm that provides a robust solution for complex nonlinear optimization problems. These algorithms also have some limitations as they require significant time and computational resources to optimize the results for all trajectories. Optimization of dynamic controller gains using the genetic algorithm in a real-world scenario requires a lot of computational resources. Furthermore, when the dynamic parameters of the system changes under various environments, these optimal gains can significantly weaken the system's flexibility and responsiveness. However, researchers have published many studies using intelligent control techniques [13], [14], [15], [16], [17], focusing on real-time obstacle avoidance and path tracking using vision-based and machine learning approaches, and using Deep Reinforcement Learning to tackle path-following problems. At last, these techniques still encounter challenges due to complex models, learning time, hardware issues, and other environmental factors.

The ultimate aim of this study is to achieve optimal performance using PID controllers, which are most commonly used in industrial and robotics applications

instead of intelligent control methods. Specifically, it aims to optimize the kinematic and dynamic controllers, proposing a method to minimize the spatial constraints and the time required to optimize the controllers. This study leverages genetic algorithms to optimize the kinematic controller, which is able to generate optimal trajectories for mobile robots with reduced time and effort, boosting practical and rapid trajectory control design. Additionally, the dynamic controller implements an adaptive PID controller [18], [19], [20], which improves the robot's ability to respond to real-time environmental changes. This method is designed to optimize the control parameters to maintain optimal control performance even if the dynamic parameters of the system change. This approach enhances the system's overall flexibility and responsiveness while ensuring stable and efficient operation in diverse settings.

This report is structured as follows. A theoretical section on a non-holonomic mobile robot system explaining the closed-loop stability equation while taking dynamic restrictions into account. Subsections of Section II also elaborates on the design of the controller using genetic algorithms and cubic polynomial interpolation for optimal kinematic controller design. The subsection also outlines the design of an adaptive PID controller as the dynamic controller to match the well-designed trajectory plan created by the kinematic controller. Section IV presents and also, compares the simulation results for the performance evaluation of the proposed controllers. Finally Section V provides the conclusion of the study.

2 System Configuration and Controller Definition

This section focuses on determining the control system of a nonholonomic mobile robot. To control a mobile robot an optimal trajectory is required. This trajectory is generated using a kinematic control technique, and optimal control is used to minimize the path-tracking error using dynamic control. We start by explaining the kinematic and dynamics of a mobile robot and then get into the details of building the controller for the same.

Fig. 1 shows the posture $([x, y, \theta]^T)$ of a mobile robot in the reference coordinate system $(X_I - Y_I)$. Here, (x, y) represents the center coordinates of the mobile robot, and θ is the angle between the robot's direction of motion and the horizontal axis. The target coordinates are $[0, 0^\circ, 0^\circ]^T$. For convenience, the target coordinate system $(X_G - Y_G)$ coincides with the reference coordinate system. θ is the angle formed by the robot's direction of motion and the horizontal axis, and ρ represents the straight-line distance between the robot's current center coordinates and the destination coordinates. α is the angle between the line represented by ρ and the robot's direction of motion, and β is the angle between ρ and the horizontal axis X. Equation (1) and (2) below represents the equation of motion for the mobile robot [4], [9], [21].

$$\begin{aligned}\alpha &= \arctan 2\left(\frac{\Delta y}{\Delta x}\right) - \theta, \\ \beta &= -(\theta + \alpha), \\ \rho &= \sqrt{(\Delta x)^2 + (\Delta y)^2}\end{aligned}\tag{1}$$

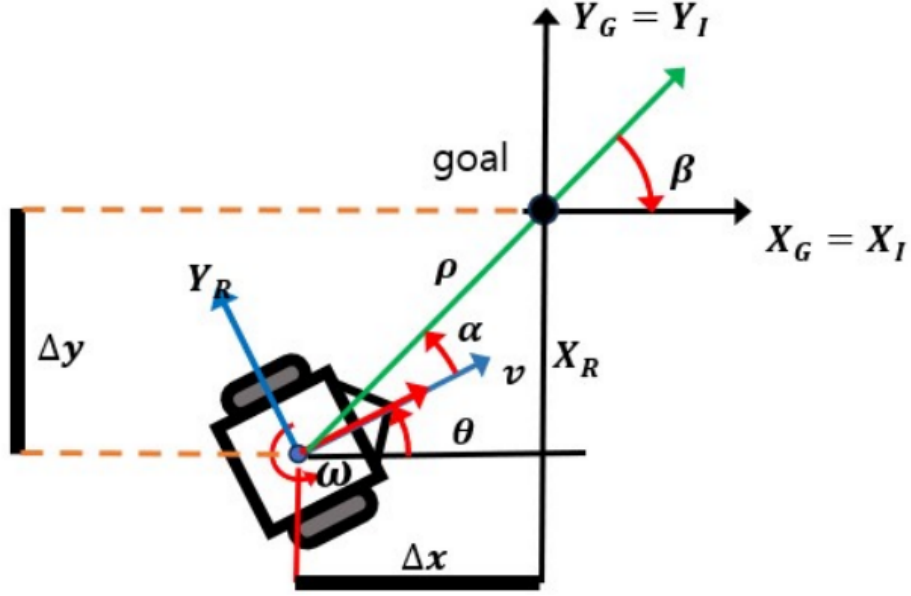


Figure 1: Robot kinematics and its frames of interest

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos \theta & 0 \\ \sin \theta & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix} \quad (2)$$

where v is the linear speed and ω is the rotational angular speed.

Robot kinematics: $\dot{x} = v \cos \theta$, $\dot{y} = v \sin \theta$, $\dot{\theta} = \omega$.

Define goal-relative quantities: $\Delta x := x_g - x$, $\Delta y := y_g - y$,

$$\rho := \sqrt{\Delta x^2 + \Delta y^2}, \quad \phi := \text{atan2}(\Delta y, \Delta x),$$

$$\alpha := \phi - \theta, \quad \beta := \theta_g - \phi.$$

(1) **Time derivative of ρ :**

$$\dot{\rho} = \frac{\Delta x \dot{\Delta x} + \Delta y \dot{\Delta y}}{\rho}.$$

Since $\dot{\Delta x} = -\dot{x} = -v \cos \theta$, $\dot{\Delta y} = -\dot{y} = -v \sin \theta$,

$$\dot{\rho} = -\frac{\Delta x v \cos \theta + \Delta y v \sin \theta}{\rho} = -v \underbrace{\frac{\Delta x \cos \theta + \Delta y \sin \theta}{\rho}}_{=\cos(\alpha)} = -v \cos \alpha.$$

(Used identity $\cos(\alpha) = \cos(\phi - \theta) = \frac{\Delta x \cos \theta + \Delta y \sin \theta}{\rho}$.)

(2) **Time derivative of ϕ :** A standard derivative of $\phi = \text{atan2}(\Delta y, \Delta x)$ gives

$$\dot{\phi} = \frac{\Delta x \dot{\Delta y} - \Delta y \dot{\Delta x}}{\rho^2}.$$

Substitute $\dot{\Delta}x = -v \cos \theta$, $\dot{\Delta}y = -v \sin \theta$:

$$\dot{\phi} = \frac{\Delta x(-v \sin \theta) - \Delta y(-v \cos \theta)}{\rho^2} = \frac{v(\Delta y \cos \theta - \Delta x \sin \theta)}{\rho^2}.$$

Recognize $\sin(\alpha) = \sin(\phi - \theta) = \frac{\Delta y \cos \theta - \Delta x \sin \theta}{\rho}$, hence

$$\boxed{\dot{\phi} = \frac{v \sin \alpha}{\rho}}.$$

(3) Time derivative of α and β : Since $\alpha = \phi - \theta$,

$$\dot{\alpha} = \dot{\phi} - \dot{\theta} = \frac{v \sin \alpha}{\rho} - \omega.$$

Since $\beta = \theta_g - \phi$ and θ_g is constant,

$$\dot{\beta} = -\dot{\phi} = -\frac{v \sin \alpha}{\rho}.$$

(4) Collecting the three derivatives:

$$\boxed{\begin{aligned} \dot{\rho} &= -v \cos \alpha, \\ \dot{\alpha} &= \frac{v \sin \alpha}{\rho} - \omega, \\ \dot{\beta} &= -\frac{v \sin \alpha}{\rho}. \end{aligned}}$$

(5) Matrix (compact) form:

$$\begin{bmatrix} \dot{\rho} \\ \dot{\alpha} \\ \dot{\beta} \end{bmatrix} = \begin{bmatrix} -\cos \alpha & 0 \\ \frac{\sin \alpha}{\rho} & -1 \\ -\frac{\sin \alpha}{\rho} & 0 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix}.$$

Points to notice:

- The derivation uses the geometric identities for $\cos(\phi - \theta)$ and $\sin(\phi - \theta)$ expressed via $\Delta x, \Delta y, \theta$.
- There is a singularity at $\rho = 0$ because terms with $1/\rho$ blow up; practical controllers switch or saturate near the goal.
- Angles should be wrapped (e.g. to $(-\pi, \pi]$) in implementations to avoid discontinuities.

If conditions $\alpha \in (-\frac{\pi}{2}, \frac{\pi}{2})$ and $\beta \in (-\pi, \pi]$ are satisfied in Fig. 1, the mobile robot moves only in the forward direction. The control signal $([v, \omega]^T)$ is chosen as

a linear controller, as shown in Equation (4), to drive the robot from its actual set position $([\rho, \alpha, \beta]^T)$ to the target position [4], [9], [21].

$$\begin{bmatrix} \dot{\rho} \\ \dot{\alpha} \\ \dot{\beta} \end{bmatrix} = \begin{bmatrix} -\cos(\alpha) & 0 \\ \sin(\alpha)/\rho & -1 \\ -\sin(\alpha)/\rho & 0 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix} \quad (3)$$

$$\begin{bmatrix} v \\ \omega \end{bmatrix} = \begin{bmatrix} K_\rho \rho \\ K_\alpha \alpha + K_\beta \beta \end{bmatrix} \quad (4)$$

where K_ρ , K_α , K_β represent the gains of the kinematic controller.

$$\begin{bmatrix} \dot{\rho} \\ \dot{\alpha} \\ \dot{\beta} \end{bmatrix} = \begin{bmatrix} -\cos(\alpha) & 0 \\ \sin(\alpha)/\rho & -1 \\ -\sin(\alpha)/\rho & 0 \end{bmatrix} \begin{bmatrix} K_\rho \rho \\ K_\alpha \alpha + K_\beta \beta \end{bmatrix} \quad (5)$$

When transforming the kinematic equation of the mobile robot with respect to $[\rho, \alpha, \beta]^T$, the closed-loop system of the mobile robot is represented by Equation (4). The system has a unique equilibrium point at $[\rho, \alpha, \beta]^T = [0, 0, 0]^T$ and solving equation (3) and (4) we get the equation (5) and can be simplified as equation (6).

$$\begin{bmatrix} \dot{\rho} \\ \dot{\alpha} \\ \dot{\beta} \end{bmatrix} = \begin{bmatrix} -K_\rho \rho \cos \alpha \\ K_\rho \sin \alpha - K_\alpha \alpha - K_\beta \beta \\ -K_\rho \sin \alpha \end{bmatrix} \quad (6)$$

The velocity and angular velocity of the mobile robot are expressed in terms of the angular velocities of both wheels of the mobile robot, as shown in Equation (7).

$$\omega_r = \frac{1}{r}(v + R\omega), \quad \omega_l = \frac{1}{r}(v - R\omega) \quad (7)$$

ω_r and ω_l represent the angular velocities of the right and left wheels of the mobile robot, respectively. R is the distance from the wheels to the center of the mobile robot, and r is the radius of the wheels.

For precise control of the mobile robot, its dynamic model must be considered. In this study, the dynamic model of a nonholonomic mobile robot was constructed as shown in Equation (8), which is taken from the previous paper [10].

$$M(q)\ddot{q} + V_m(q, \dot{q})\dot{q} + F(\dot{q}) + G(q) + d = B(q) - A^T(q)\lambda \quad (8)$$

where, q is $[x, y, \theta]^T$ and $M(q)$ is the positive definite symmetric inertia matrix, $V_m(q, \dot{q})$ is the centripetal and Coriolis matrix, $F(\dot{q})$ represents surface friction, $G(q)$ is the gravity vector, and d is bounded by unknown disturbances. $B(q)$ represents the input transformation matrix, represents the input torque, $A(q)$ represents the constraint matrix, and λ is the constraint force vector. The friction force ($F(\dot{q}) = -(D + C)\dot{q}$) includes viscous friction ($-D\dot{q}$) and ground friction ($-C\dot{q}$) and is modeled so that the mobile robot can reflect its movements in a real environment [23], [24], [25]. Since the movement of the mobile robot is restricted to the horizontal plane, gravity is set to $G(q) = 0$.

The physical movement of the mobile robot determines the maximum velocity and maximum angular velocity of the controlled mobile robot. Therefore, the control

commands for the mobile robot must adhere to physical constraints. Consequently, the stability condition of the closed-loop system for the linear controller can be defined as shown in Equation (9). This equation (8) represents the closed-loop stability condition from Siegwart et al. [21] and the closed-loop stability condition considering the dynamics of the mobile robot as proposed by Park et al. [4]. Therefore, the stability of the mobile robot system is assured when we select the kinematic control gains by the stability condition specified in Equation (8).

$$\begin{aligned}
0 < K_\rho &\leq \frac{v_{Max}}{\rho_{Max}} \\
K_\beta &< 0 \\
K_\alpha + \frac{5}{3}K_\beta - \frac{2}{\pi}K_\rho &> 0 \\
\frac{\pi}{2}K_\alpha - \pi K_\beta &\leq \omega_{Max}
\end{aligned} \tag{9}$$

ρ_{Max} is the maximum straight-line distance between the center coordinate of the robot and the destination coordinate, v_{Max} defines the maximum velocity of the mobile robot, and ω_{Max} represents the maximum angular velocity of the mobile robot.

For controlling the mobile robot, selecting linear kinematic control gains within the range of the stability condition in Equation (9) allows the robot to reach the target point. However, these selected gains are not necessarily the optimal control gains for the mobile robot. This is a complex nonlinear control problem that is difficult to solve using general optimal control theory [4], [9], [21]. The first row of Equation (6) is a linear time-varying differential equation concerning ρ , where $\alpha(t)$ is a time-varying parameter. Linear differential equations exhibit a consistent pattern of solutions only concerning the initial value $\rho(0)$, regardless of whether they involve time-varying or time-invariant elements. The second and third rows of Equation (6) are nonlinear differential equations concerning $\rho(t)$ and $\beta(t)$, which can yield entirely different results depending on the magnitudes of the initial values $\alpha(0)$ and $\beta(0)$. Therefore, optimal control gains must be derived to reach the target point quickly and accurately.

In mobile robot control, studies have shown that optimal trajectories generated using kinematic controllers when combined with dynamic techniques have enabled robots to follow the generated trajectory tightly with reduced path-tracking error. Recently, the study by Park et al. [4] laid the foundation for addressing issues inherent in control techniques composed of individual controllers by integrating these two methods. They derived a closed-loop stability condition for gains that ensure system stability by considering the dynamics of the mobile robot, addressing command values for kinematic speed and angular velocity that exceed physical limits. They utilized genetic algorithms to determine optimal gains from dynamic controllers. This led to stable and predictable operations of robots, enhancing their ability to adapt to environmental changes, extending their lifespan. However, with fixed gains a dynamic controller struggles to adapt to variations in system parameters, like change in the mass of the robot. Also, finding optimal gains for dynamic controllers using genetic algorithms requires computational resources and effort. Therefore, in the approach using gradient descent method for calculating gains for dynamic controllers, the gains are adaptive and adapt in response to variation in system

parameters.

The overall controller design of a mobile robot involves 2 approaches. First, optimize the kinematic controller gains and use the optimal gains for generating the robot's optimal trajectory. The next step is to reduce the tracking error of the mobile robot by using an adaptive PID controller. The first subsection will explain the design of controllers using genetic algorithms and cubic interpolation for optimal kinematic controller design. The second subsection will cover the design of an adaptive PID controller as a dynamic controller aligned with a well-designed trajectory plan.

2.1 Design of Kinematic Controller

The kinematic controller is a crucial part as it is used to generate optimal paths for the robot. The controller gains are selected as shown in equation (9) such that the robot can reach the goal pose without exceeding the physical limits of linear and angular velocities. However, gains may not be optimal as they involve solving highly complex nonlinear control problems, which is challenging when solved using general optimal control theory. In this study, to determine the optimal kinematic controller gains that allow the robot to reach the target pose most quickly and efficiently, genetic algorithms are used. Genetic Algorithms(GA) [26], [27] are widely recognized as global search algorithms.

In simple terms, GA is a population-based optimization method that finds the optimal solution using a fitness function, then repeatedly evolves the population by selecting the high-fitness individuals from the population. It also recombines them through crossover and also, uses mutation to explore new possibilities. These are particularly useful where traditional calculus-based optimizers fail for complex, non-linear and discrete problems like this one. Lets understand this with an example, let Alex be a candidate who need to cross the park with obstacles like a rock on the floor and birds above him. He has 4 possible actions to take, jump ($\uparrow$), crouch($\downarrow$), move left($\leftarrow$) or move right($\rightarrow$) we can call them genes. Now the algorithm starts with a random population with 12 random action sequences that Alex can take. These individual sequences can be named DNA sequences or different versions of Alex. After running through the first iteration, i.e. all the sequences are executed to find how far Alex reaches on the path. A fitness function is used to measure the fitness of different versions of Alex based on how far they reach. The GA now selects the best 2 sequences, for example and then applies crossover, i.e. mixing the sequences to generate a new population with the features of the best 2 survivors, but also adding mutation such that some genes of this new generation are altered so that crucial features are not lost. Suppose the fittest versions of Alex did not use the jump gene, so it will not be passed onto the next generation, and hence Alex might get stuck in the same place, which we refer as a local minima. So, mutation helps avoid this and also explore new possibilities. Thus, doing the same process over many generations leads to generating a sequence for Alex that is efficient to navigate through the park instead of randomly guessing. All in all, GA combines all, selection, crossover and mutation, to produce optimal fitness, which is what we use to generate optimal trajectory gains.

The design of kinematic controllers is independent of the dynamics of the mobile robot. Therefore, Fig. 2 implements the kinematic control system using only the kinematic equations of the mobile robot. Of course, if the conditions of the closed-loop stability equation (9) are satisfied, stable control performance will be achieved. However, since Equation (6) is a nonlinear differential equation, the convergence trajectory $[\rho(t), \alpha(t), \beta(t)]^T$ can take very different forms depending on the initial conditions $[\rho(0), \alpha(0), \beta(0)]^T$, even with the same control gains $[K_\rho, K_\alpha, K_\beta]^T$. Consequently, the convergence speed also varies. Thus, the minimum-time control gain for the given initial conditions $[\rho(0), \alpha(0), \beta(0)]^T$ is determined uniquely based on these initial conditions, illustrating the necessity of adapting control gains to specific scenarios.

Therefore, $[K_\rho, K_\alpha, K_\beta]^T$ is considered as the chromosome(or gene), or search variable, in the genetic algorithm. By numerically solving the nonlinear differential Equation (6) based on these search variable values, a fitness function is designed using the time T_f it takes to converge to the target point $[\rho, \alpha, \beta]^T = [0, 0^\circ, 0^\circ]^T$ and the performance error to determine the optimal gains. Equation (10) represents the fitness function of the genetic algorithm. The fitness function is set to ensure that the mobile robot achieves the fastest and most accurate posture at the target point.

$$F = aT_f + b(|\rho(T_f)| + |\alpha(T_f)| + |\beta(T_f)|) \quad (10)$$

where T_f is the final target arrival time, and $\alpha(T_f)$ and $\beta(T_f)$ are the posture angles of the mobile robot at T_f . The parameters a and b represent weights.

Although $\alpha(0)$ and $\beta(0)$ can be subdivided into infinitely many segments within the intervals $(-\pi/2, \pi/2]$ and $(-\pi, \pi]$, respectively, finding the optimal controller gains $[K_\rho, K_\alpha, K_\beta]^T$ would be computationally infeasible with infinite subdivisions. Therefore, $\alpha(0)$ and $\beta(0)$ intervals are discretized appropriately to generate optimal gains. Thus, as shown in Equation (11), $\alpha(0)$ and $\beta(0)$ are discretized into N_α and N_β points, respectively, to create grid points.

$$\begin{aligned} \alpha(0) &\in \left\{ -\frac{\pi}{2} + \frac{\pi}{N_\alpha}, -\frac{\pi}{2} + \frac{2\pi}{N_\alpha}, \dots, -\frac{\pi}{2} + \frac{N_\alpha\pi}{N_\alpha} \right\}, \\ \beta(0) &\in \left\{ -\pi + \frac{2\pi}{N_\beta}, -\pi + \frac{2(2\pi)}{N_\beta}, \dots, -\pi + \frac{N_\beta(2\pi)}{N_\beta} \right\}. \end{aligned} \quad (11)$$

The process can be expressed in algorithmic form as follows.

1. Initialize: $\rho(0) = 5$, $\alpha(0) = -\frac{\pi}{2} + \frac{\pi}{N_\alpha}$
 - 1-1) Set $\beta(0) = -\pi + \frac{2\pi}{N_\beta}$
2. In the genetic algorithm, set the kinematic control gains $[K_\rho, K_\alpha, K_\beta]^T$ as search parameters and ensure that the stability condition (9) is satisfied. Solve the nonlinear differential Equation (6) numerically from the initial value $[\rho(0), \alpha(0), \beta(0)]^T$, and calculate the fitness function of Equation (10).
3. Run the genetic algorithm until the fitness function is minimized to find the optimal gains $[K_\rho, K_\alpha, K_\beta]^T$ at the initial condition.

4. If $\beta(0) = \pi$, proceed to Step 5. Otherwise, set $\beta(0) \leftarrow \beta(0) + \frac{2\pi}{N_\beta}$ and return to Step 2.
5. If $\alpha(0) = \frac{\pi}{2}$, terminate the process. Otherwise, set $\alpha(0) \leftarrow \alpha(0) + \frac{\pi}{N_\alpha}$ and return to Step 1-a.

The above algorithm uses the genetic algorithm for each grid point to find the control gains $[K_\rho, K_\alpha, K_\beta]^T$ that minimize the fitness value while satisfying the closed-loop stability condition. For initial postures of the mobile robot that are not discretized, an interpolation algorithm can generate a gain value. Fig. 3 schematically illustrates the process of generating optimal gains for discretized initial postures using the genetic algorithm and generating gain values for arbitrary postures using a cubic interpolation algorithm.

One of the representative algorithms for cubic polynomial interpolation is Cubic Spline Interpolation. Cubic Spline Interpolation is a method that fits a smooth curve through a set of data points. This models each segment with a cubic polynomial to smoothly connect the entire set of data points. In this algorithm, the initial posture angle data $[\alpha(0), \beta(0)]^T$ of the mobile robot are selected as the x-y axis sample grid points, and the kinematic linear control gains $[K_\rho, K_\alpha, K_\beta]^T$ are used as the sample values to perform two-dimensional cubic interpolation. The basis functions used in cubic interpolation are cubic polynomials for each axis direction, as described in Equation (12). The nodes for interpolation are $(N_\alpha - 1)$ and $(N_\beta - 1)$ based on the grid intervals of $\alpha(0)$ and $\beta(0)$.

$$\begin{aligned}
&\text{Each segment } [\alpha_i, \alpha_{i+1}] : \quad S_i(\alpha) = \\
&\quad a_i(\alpha - \alpha_i)^3 + b_i(\alpha - \alpha_i)^2 + c_i(\alpha - \alpha_i) + d_i, \\
&\quad \text{Each segment } [\beta_i, \beta_{i+1}] : \quad S_i(\beta) = \\
&\quad e_i(\beta - \beta_i)^3 + f_i(\beta - \beta_i)^2 + g_i(\beta - \beta_i) + h_i,
\end{aligned} \tag{12}$$

where α_i and β_i represent the initial posture angles of the mobile robot, while $a_i, b_i, c_i, d_i, e_i, f_i, g_i, h_i$ denote the coefficients of the cubic polynomials.

The optimized control gains do not differ significantly from adjacent optimal gains, so using a cubic interpolation algorithm alone has little impact on the control performance of the mobile robot. For interpolation, the input variables $(\alpha(0) \ \beta(0))^T$ are used, and the variables $(K_\rho, K_\alpha, K_\beta)^T$ for interpolation are each represented by a cubic polynomial. Values modeled by cubic polynomials provide smoother and more accurate results than those obtained through linear interpolation.

2.2 Design of Dynamic Controller for Mobile Robot using Gradient Descent Learning

The dynamic controller is implemented using a PID controller with adaptive gains, which are updated in real-time using the Gradient Descent method. In this setup, the PID controllers receive the commanded angular velocities of the left and right wheels, generated by the kinematic controller as their reference setpoints. They

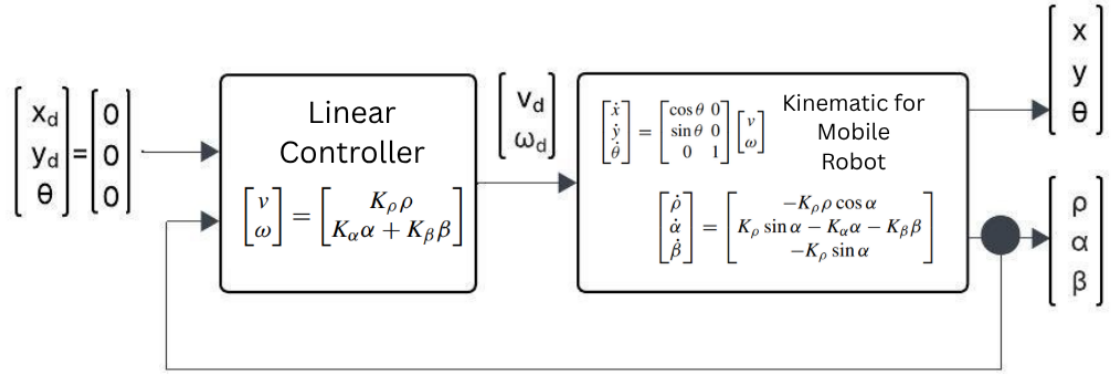


Figure 2: Kinematic Control for Mobile Robot

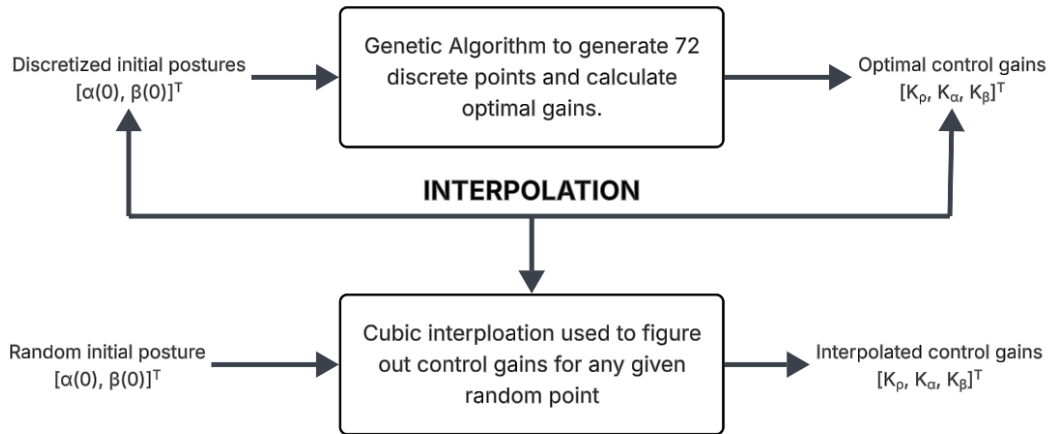


Figure 3: GA-based cubic polynomial interpolation.

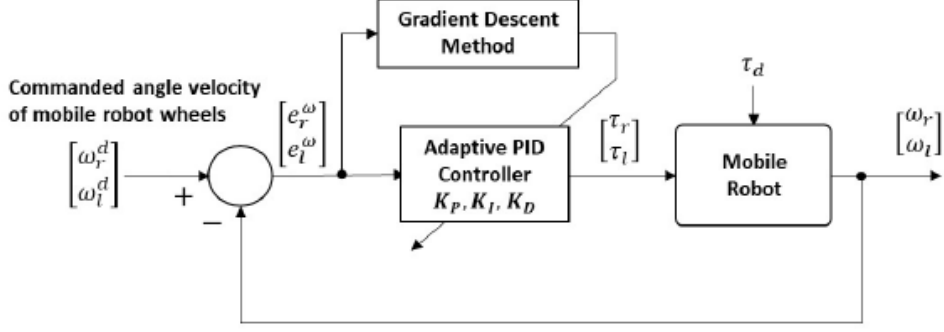


Figure 4: Dynamic control for a mobile robot.

continuously measure the actual angular velocities of both wheels to determine the tracking error, which reflects the error between the desired and actual motion.

This tracking error is then fed into the Gradient Descent algorithm to adjust the PID gains (proportional, integral, and derivative) continuously, ensuring that the controller can adapt to changing system dynamics or external disturbances. Using these updated gains, the PID controller calculates the control output, which is the torque applied to each wheel. The calculated torque is subsequently incorporated into the dynamic equations of the system (Equation (1)), which models the relationship between applied torque, wheel inertia, friction, and other dynamic effects. These equations also account for potential torque disturbances, enabling the controller to maintain accurate wheel velocities even under external influences or unexpected load variations. By continuously updating the PID gains based on real-time feedback, the system achieves robust and precise tracking performance. The equation for basic PID controller is given in equation (1)

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (1)$$

Where, $u(t)$ is the control output, $e(t)$ is the error signal (difference between setpoint and measured value), K_p , K_i , and K_d are the proportional, integral, and derivative gains, respectively. Our PID controller consists of adaptive value for gains too. So our equation will be (2).

$$\tau_j(t) = (K_P + dK_1)e_j(t) + (K_I + dK_2) \int_0^t e_j(\tau) d\tau + (K_D + dK_3)\dot{e}_j(t) \quad (2)$$

Where $\tau_j(t)$ is the control output, which represents the torque of the left and right wheels. $e(t)$ is the error signal (the difference between the setpoint and the measured angular velocity of the wheel), K_p , K_i , and K_d are the proportional, integral, and derivative gains, respectively. dK_1 , dK_2 , and dK_3 are the adaptive gains for the proportional, integral, and derivative gains, respectively.

K_p , K_i , and K_d will be fixed initially to a stable set of values. So now to calculate the PID output we only need the values of dK_1 , dK_2 , and dK_3 . To calculate this values lets first understand the energy error functions used in control systems and robotics.

An energy error function is commonly used to quantify the discrepancy between the desired and actual energy of a system. Unlike conventional error measurement that focus solely on position or velocity deviations, the energy error function provides a comprehensive metric that captures the dynamic performance of the system. By minimizing this energy error, controllers can be designed to achieve not only accurate trajectory tracking but also improved efficiency, stability, and robustness against disturbances and external perturbations. A general form of the energy error function can be expressed as:

$$E_e = \frac{1}{2} (E_d - E)^2 \quad (3)$$

where E_d is the desired energy of the system, E is the actual energy, and E_e represents the energy error. This function can be applied to both kinetic and potential energy components, depending on the specific system dynamics. Equation (3) shows the formulation of the error function $E_p(k)$ in our case in form of kinetic energy, which is dependent on the angular velocities of the wheels:

$$E_p(k) = \frac{1}{2} \sum_{j=1}^2 [q_j^d(k) - q_j(k)]^2 = \frac{1}{2} \sum_{j=1}^2 [e_j(k)]^2 \quad (4)$$

where

$$\mathbf{q}^d(k) = S \begin{bmatrix} q_1^d(k) \\ q_2^d(k) \end{bmatrix} = \begin{bmatrix} \omega_r^d(k) \\ \omega_l^d(k) \end{bmatrix}$$

are the target angular velocity values for the right and left wheels of the mobile robot, and

$$\mathbf{q}(k) = \begin{bmatrix} q_1(k) \\ q_2(k) \end{bmatrix} = \begin{bmatrix} \omega_r(k) \\ \omega_l(k) \end{bmatrix}$$

are the measured angular velocity values. Therefore, $e_j(k)$ represents the angular velocity error of the mobile robot's wheels at sampling time k .

We will select the gradient as a response of Error function with respect to individual adaptive gains dK_1, dK_2, dK_3 .

Let

$$\frac{\partial E_p}{\partial dK_i} \approx \frac{E_p(k+1) - E_p(k)}{dK_i(k+1) - dK_i(k)}.$$

This equation shows the gradient as the rate of change of the error with respect to a change in the adaptive gain.

Case 1: If the Gradient is Positive

If

$$\frac{\partial E_p}{\partial dK_i} > 0,$$

Increase in gain means error will also increase considering the gradient to be positive.

Decrease in gain means error will also decrease considering the gradient to be positive.

Case 2: Gradient is Negative

If

$$\frac{\partial E_p}{\partial dK_i} < 0,$$

Increase in gain means error will decrease considering the gradient to be negative.

Decrease in gain means error will increase considering the gradient to be negative.

Conclusion

- If the gradient is positive, the next gain value should be decreased to reduce the error.
- If the gradient is negative, the next gain value should be increased to reduce the error.

Hence, it should behave inversely, therefore we will incorporate the negative sign for gradient. The gradient can be represented by $\Delta K_1, \Delta K_2, \Delta K_3$ for the changes in

dK_1, dK_2, dK_3 adaptive gains of the PID controller as follows:

$$\Delta K_i = -\zeta_i \frac{\partial E_p(k)}{\partial dK_i(k)} \quad (5)$$

where ζ_i represents the learning rate, $i = 1, 2, 3$, and $j = 1, 2$. $\zeta_i > 0$. The subtraction ensures that the gain is always updated in the direction that reduces the error, regardless of the sign of the gradient. If the gradient is positive, the gain decreases; if the gradient is negative, the gain increases. This allows the adaptive PID controller to adjust its gains automatically in real time, ensuring improved tracking performance and robustness against disturbances.

We will convert our equation as a chain of derivative which will be easy to calculate:

$$\Delta K_i = -\zeta_i \frac{\partial E_p(k)}{\partial dK_i(k)} = -\zeta_i \frac{\partial E_p(k)}{\partial q_j(k)} \frac{\partial q_j(k)}{\partial \tau_j(k)} \frac{\partial \tau_j(k)}{\partial dK_i(k)} \quad (6)$$

The first part of the chain is:

$$\frac{\partial E_p(k)}{\partial q_j(k)} = -(q_j^d(k) - q_j(k)) = -e_j(k)$$

Proof :

$$\frac{\partial E_p(k)}{\partial q_1(k)} = \frac{\partial}{\partial q_1(k)} \left[\frac{1}{2} (q_1^d(k) - q_1(k))^2 + \frac{1}{2} (q_2^d(k) - q_2(k))^2 \right]$$

Since the second term does not depend on $q_1(k)$, its derivative is zero:

$$\frac{\partial E_p(k)}{\partial q_1(k)} = \frac{\partial}{\partial q_1(k)} \left[\frac{1}{2} (q_1^d(k) - q_1(k))^2 \right]$$

Apply the chain rule:

$$\begin{aligned}
&= \frac{1}{2} \cdot 2(q_1^d(k) - q_1(k)) \cdot \frac{\partial}{\partial q_1(k)}(q_1^d(k) - q_1(k)) \\
&= (q_1^d(k) - q_1(k))(-1) \\
&= -(q_1^d(k) - q_1(k))
\end{aligned}$$

Thus, in general:

$$\frac{\partial E_p(k)}{\partial q_j(k)} = -(q_j^d(k) - q_j(k)) = -e_j(k)$$

The second part of the chain:

$$\frac{\partial q_j(k)}{\partial \tau_j(k)}$$

represents the Jacobian of the mobile robot system, which is difficult to determine precisely. In practical control systems, this value cannot be known exactly. Therefore, the mathematical approximation $(\Delta q_j(k)/\Delta \tau_j(k))$ is used instead.

Moreover, due to the nature of the gradient descent learning method, the sign of this value is more important than its actual magnitude. Hence, it is modified as shown in Equation (14):

$$\frac{\partial q_j(k)}{\partial \tau_j(k)} := \text{sgn}\left(\frac{\Delta q_j(k)}{\Delta \tau_j(k)}\right).$$

This approach is also employed in simulation experiments to reflect conditions that may arise in real-world implementations.

The third part of the chain:

$$\frac{\partial \tau_j(k)}{\partial dK_i(k)} \tag{7}$$

To determine the third part we will convert the equation (2) of the PID controller in discrete form

For discrete systems PID function can be written as

$$\begin{aligned}
\tau_j(k) &= (K_p + dk_1) e_j(k) \\
&\quad + (K_I + dk_2) T_s \sum_{i=1}^k e_j(i) \\
&\quad + \frac{(K_d + dk_3)}{T_s} [e_j(k) - e_j(k-1)] \\
\tau_j(k-1) &= (K_p + dk_1) e_j(k-1) \\
&\quad + (K_I + dk_2) T_s \sum_{i=1}^{k-1} e_j(i) \\
&\quad + \frac{(K_d + dk_3)}{T_s} [e_j(k-1) - e_j(k-2)]
\end{aligned} \tag{8}$$

$$\begin{aligned}
\tau_j(k) - \tau_j(k-1) &= (K_p + dk_1) [e_j(k) - e_j(k-1)] \\
&+ (K_I + dk_2) T_s \left(\sum_{i=1}^k e_j(i) - \sum_{i=1}^{k-1} e_j(i) \right) \\
&+ \frac{(K_d + dk_3)}{T_s} \left([e_j(k) - e_j(k-1)] \right. \\
&\quad \left. - [e_j(k-1) - e_j(k-2)] \right)
\end{aligned} \tag{9}$$

Therefore

$$\begin{aligned}
\tau_j(k) &= \tau_j(k-1) + (K_p + dk_1)(e_j(k) - e_j(k-1)) + (k_i + dk_2)e_j(k) \\
&+ (K_{d1} + dk_3)(e_j(k) - 2e_j(k-1) + e_j(k-2))
\end{aligned} \tag{10}$$

Here, the author has taken the sampling time to be one second therefore, the T_s seen in equation (9) cannot be seen in the final form of the equation (10).

From Equation (10),

$\frac{\partial \tau_j(k)}{\partial dK_i(k)}$ is derived as follows:

$$\frac{\partial \tau_j(k)}{\partial dK_1(k)} = e_j(k) - e_j(k-1), \tag{11}$$

$$\frac{\partial \tau_j(k)}{\partial dK_2(k)} = e_j(k), \tag{12}$$

$$\frac{\partial \tau_j(k)}{\partial dK_3(k)} = e_j(k) - 2e_j(k-1) + e_j(k-2). \tag{13}$$

Now we have all the functions in generalized for to calculate the change in the adaptive gain $\Delta dK_i(k)$

$$dK_i(k+1) = dK_i(k) + \Delta dK_i(k)$$

Thus, we will use these equations to create the output of the Gradient Descent Method, which will be fed to the PID controller to calculate the τ_l and τ_r of the wheels.

$$\Delta dK_1(k) = -e_j(K) * \text{sgn}\left(\frac{\Delta q_j(k)}{\Delta \tau_j(k)}\right) * e_j(k) - e_j(k-1) \tag{14}$$

$$\Delta dK_2(k) = -e_j(K) * \text{sgn}\left(\frac{\Delta q_j(k)}{\Delta \tau_j(k)}\right) * e_j(k) \tag{15}$$

$$\Delta dK_3(k) = -e_j(K) * \text{sgn}\left(\frac{\Delta q_j(k)}{\Delta \tau_j(k)}\right) * e_j(k) - 2e_j(k-1) + e_j(k-2) \tag{16}$$

We will be using these equation in Matlab for the Gradient Descent Method for adaptive Gain generation, which will be fed to the adaptive PID controller containing the PID equation (2).

We will now check the system stability using the Lyapunov function. The Lyapunov function should be positive, and its derivative should be negative.

Let Lyapunov function be:

$$V(e, \tilde{K}) = \frac{1}{2}e^T e + \frac{1}{2}\tilde{K}_p^T \Sigma_p^{-1} \tilde{K}_p + \frac{1}{2}\tilde{K}_i^T \Sigma_i^{-1} \tilde{K}_i + \frac{1}{2}\tilde{K}_d^T \Sigma_d^{-1} \tilde{K}_d.$$

Taking derivative of V :

$$\frac{d}{dt}(e^T e) = \dot{e}^T e + e^T \dot{e} = 2e^T \dot{e} \quad \Rightarrow \quad \frac{1}{2} \frac{d}{dt}(e^T e) = e^T \dot{e}.$$

Similarly,

$$\frac{d}{dt} \left(\frac{1}{2} \tilde{K}_p^T \Sigma_p^{-1} \tilde{K}_p \right) = \tilde{K}_p^T \Sigma_p^{-1} \dot{\tilde{K}}_p.$$

Thus,

$$\dot{V} = e^T \dot{e} + \tilde{K}_p^T \Sigma_p^{-1} \dot{\tilde{K}}_p + \tilde{K}_i^T \Sigma_i^{-1} \dot{\tilde{K}}_i + \tilde{K}_d^T \Sigma_d^{-1} \dot{\tilde{K}}_d.$$

Given the adaptive law:

$$\dot{\tilde{K}}_p = \Delta \tilde{K}_p = -\zeta \frac{\partial E_p(K)}{\partial K_p(K)},$$

$$\Sigma_p^{-1} \dot{\tilde{K}}_p = \Sigma_p^{-1} \Delta \tilde{K}_p = -\frac{\partial E_p(K)}{\partial K_p(K)}.$$

Substituting in the equation, we get

$$\begin{aligned} \dot{V} &= e^T \dot{e} - \tilde{K}_p^T \frac{\partial E_p(K)}{\partial K_1} - \tilde{K}_i^T \frac{\partial E_p(K)}{\partial K_2} - \tilde{K}_d^T \frac{\partial E_p(K)}{\partial K_3}. \\ &= e^T \dot{e} - K_p e \dot{e} - K_i e \int e dt - K_d \dot{e}. \end{aligned}$$

Thus, we have checked the stability of the adaptive PID controller. The adaptive PID controller provides Lyapunov stability for the mobile robot system through gradient descent learning. This approach allows the system to adapt to various environmental changes and adjust the control gains in real time to minimize trajectory error, maintaining stability through error reduction without explicitly relying on dynamic equations.

The output of the PID controller will now be used to calculate the angular velocities required for the wheels using the dynamic equation below

$$M(q) \ddot{q} + V_m(q, \dot{q}) \dot{q} + F(\dot{q}) + G(q) + \tau_d = B(q) \tau - A^T(q) \lambda \quad (17)$$

We will not go through the derivation of equation (17), as it is extensive enough to constitute an entire journal article on its own. The equation (17) is based on the journal reference [9].

Here $q = [x, y, \theta]^T$, and $M(q)$ is the positive definite symmetric inertia matrix. $V_m(q, \dot{q})$ denotes the centripetal and Coriolis matrix, $F(\dot{q})$ represents the surface friction, $G(q)$ is the gravity vector, and τ_d is the bounded unknown disturbance. The matrix $B(q)$ represents the input transformation matrix, τ represents the input torque, $A(q)$ denotes the constraint matrix, and λ is the constraint force vector.

The friction force,

$$F(\dot{q}) = -(D + C)\dot{q},$$

includes viscous friction $-D\dot{q}$ and ground friction $-C\dot{q}$, and is modeled so that the mobile robot can accurately reflect its motion in a real environment. Since the movement of the mobile robot is restricted to the horizontal plane, gravity is set to $G(q) = 0$.

$$M(q) = \begin{bmatrix} m & 0 & md \sin \theta \\ 0 & m & -md \cos \theta \\ md \sin \theta & -md \cos \theta & I \end{bmatrix}$$

$$V_m(q, \dot{q}) = \begin{bmatrix} 0 & 0 & md \dot{\theta} \cos \theta \\ 0 & 0 & md \dot{\theta} \sin \theta \\ 0 & 0 & 0 \end{bmatrix}$$

$$B(q) = \frac{1}{r} \begin{bmatrix} \cos \theta & \cos \theta \\ \sin \theta & \sin \theta \\ R & -R \end{bmatrix}$$

$$F(\dot{q}) = -(D + C) \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix}$$

$$D = \begin{bmatrix} k_{d1}m + c_1 & 0 & 0 \\ 0 & k_{d1}m + c_1 & 0 \\ 0 & 0 & k_{d2}m + c_2 \end{bmatrix}$$

$$C = \begin{bmatrix} k_{c1}m + e_1 & 0 & 0 \\ 0 & k_{d1}m + e_1 & 0 \\ 0 & 0 & k_{d2}m + e_2 \end{bmatrix}$$

$$A(q)^T = \begin{bmatrix} -\sin \theta \\ \cos \theta \\ -d \end{bmatrix}$$

$$\lambda = -m(\dot{x}_c \cos \theta + \dot{y}_c \sin \theta) \dot{\theta}$$

The final control system for the robot is explained using the following control diagram for better understanding.

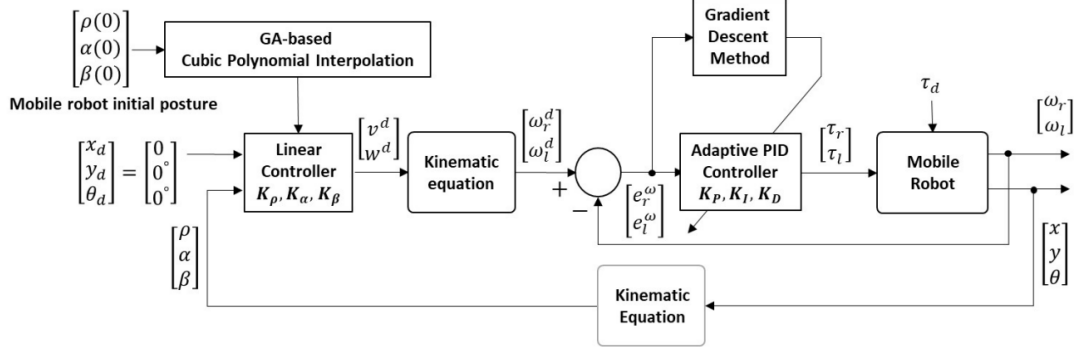


Figure 5: Control system for the mobile robot.

3 MATLAB Implementation, Results and Comparisons

We now have all the mathematical formulations required for implementing this system in MATLAB.

3.1 Implementation of Genetic Algorithm Table and Its Comparison

We consider the same grid points used by the author. The initial posture angles are chosen as follows:

$$\alpha \in [-60^\circ, 90^\circ], \quad \beta \in [-150^\circ, 180^\circ]$$

A total grid of $6 \times 12 = 72$ points is generated, with 6 uniformly spaced values of α and 12 uniformly spaced values of β .

Using these grid points, we generate optimal gain values using the genetic algorithm (GA) based on the fitness function given in Equation (10) and the constraints defined in Equation (9). The MATLAB **Global Optimization Toolbox** is used to compute the gains:

$$K_\rho, \quad K_\alpha, \quad K_\beta.$$

The genetic algorithm parameters are chosen as: - Population size = 50 - Generations = 50 - Mutation rate = 0.1 - Crossover fraction = 0.8

The fitness function in Equation (10) ensures that the mobile robot reaches the target posture in the fastest and most accurate manner.

We compare the table generated by the author with the table produced using our implementation.

Author's GA table

Case	Initial Position		Optimal control gains, traveling time, and fitness				
	α_0	β_0	K_ρ	K_α	K_β	$T_f(\text{sec.})$	$F(\text{fitness})$
1	-60°	-150°	0.5178	1.1583	-0.2721	13.73	13.7590
2	-60°	-120°	0.5180	1.1079	-0.2254	13.17	13.1976
3	-60°	-90°	0.5180	0.9407	-0.0856	12.76	12.7894
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
49	60°	-150°	0.5180	1.2774	-0.3908	14.00	14.0320
50	60°	-120°	0.5174	1.3259	-0.4234	13.40	13.4262
51	60°	-90°	0.5180	1.3990	-0.4624	12.88	12.9148
52	60°	-60°	0.5178	1.5180	-0.5192	12.50	12.5436
53	60°	-30°	0.5179	1.6097	-0.5594	13.27	12.3103
54	60°	0°	0.5178	1.5842	-0.5735	12.21	12.2331
55	60°	30°	0.5180	1.8647	-0.4418	12.20	12.2238
56	60°	60°	0.5179	1.4650	-0.3818	12.45	12.4813
57	60°	90°	0.5180	1.2337	-0.2717	12.76	12.7996
58	60°	120°	0.5170	1.1205	-0.2341	13.18	13.2068
59	60°	150°	0.5169	1.2411	-0.3242	13.80	13.8317
60	60°	180°	0.5179	1.1138	-0.2571	14.37	14.4134
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
70	90°	120°	0.5178	1.1402	-0.2159	13.47	13.5136
71	90°	150°	0.5178	1.0825	-0.2109	14.02	14.0511
72	90°	180°	0.5179	1.1328	-0.2550	14.68	14.7120

Figure 6: Control gains by cubic polynomial interpolation algorithm by the author. (a) K_ρ (b) K_α (c) K_β .

GA Table Generated by Us

Initial_Alpha_Deg	Initial_Beta_Deg	Optimal_Kp	Optimal_Ka	Optimal_Kb	Optimal_Fitness
-60	-150	0.519	1.3118	-0.3864	14.0702
-60	-120	0.4211	1.1474	-0.2524	17.5815
-60	-90	0.519	1.2785	-0.3371	12.9715
-60	-60	0.519	0.8322	0.1362	12.6364
-60	-30	0.519	1.3578	0.2064	12.3387
-60	0	0.519	2.9356	0.093	14.8947
-60	30	0.519	2.783	0.0167	20.0479
-60	60	0.519	1.5295	-0.6062	12.6805
-60	90	0.519	1.4413	-0.5216	13.094
-60	120	0.519	1.354	-0.4452	13.6287
-60	150	0.4194	1.3025	-0.403	18.3269
-60	180	0.4224	1.0655	-0.2421	20.2391
-30	-150	0.519	1.2944	-0.3805	13.8875
-30	-120	0.4191	1.338	-0.4127	17.0215
-30	-90	0.5101	0.4475	0.0695	14.3132
-30	-60	0.519	1.3984	-0.4486	12.4456
-30	-30	0.519	1.4416	-0.4301	12.2124
-30	0	0.519	1.1148	-0.302	12.3264
-30	30	0.519	2.7832	0.0168	18.8583
-30	60	0.519	1.4497	-0.5263	12.4976
-30	90	0.519	1.3266	-0.4198	12.8983
-30	120	0.519	1.129	-0.2778	13.5361
-30	150	0.519	1.2603	-0.3677	13.9983
-30	180	0.519	1.2544	-0.3629	14.7046
0	-150	0.5167	0.6653	-0.063	16.3871
0	-120	0.519	1.1713	-0.2968	13.2849
0	-90	0.519	1.2381	-0.3425	12.7891
0	-60	0.4198	1.4815	-0.5464	15.5687
0	-30	0.519	1.2308	-0.3349	12.195
0	0	0.519	2.0374	2.9999	12.0745
0	30	0.519	1.4605	-0.5253	12.1665
0	60	0.519	1.2306	-0.3357	12.4353
0	90	0.519	1.3388	-0.4211	12.7734
0	120	0.4199	1.2606	-0.3608	17.1724
0	150	0.519	0.9205	-0.155	14.3232
0	180	0.4205	1.0266	-0.2097	19.9877

Initial_Alpha_Deg	Initial_Beta_Deg	Optimal_Kp	Optimal_Ka	Optimal_Kb	Optimal_Fitness
30	-150	0.4201	1.224	-0.3412	18.2701
30	-120	0.4219	1.34	-0.4302	17.0579
30	-90	0.519	1.28	-0.3832	12.9185
30	-60	0.4204	1.4964	-0.57	15.6476
30	-30	0.519	1.4403	-0.522	12.2607
30	0	0.519	1.5203	-0.6146	12.1536
30	30	0.519	1.5077	-0.503	12.2132
30	60	0.519	1.2798	-0.3455	12.4456
30	90	0.519	1.1958	-0.2949	12.8066
30	120	0.519	1.1293	-0.2559	13.2853
30	150	0.519	1.1515	-0.2752	13.8418
30	180	0.519	1.1625	-0.2854	14.5077
60	-150	0.4206	1.2929	-0.3956	18.3001
60	-120	0.519	1.2703	-0.3792	13.6749
60	-90	0.519	1.1568	-0.3013	13.3789
60	-60	0.519	1.4748	-0.5537	12.6976
60	-30	0.519	1.7831	-0.4047	13.8615
60	0	0.519	1.5219	-0.6138	12.3459
60	30	0.519	1.6463	-0.3348	12.3374
60	60	0.42	1.276	-0.2818	16.0573
60	90	0.519	1.0756	-0.179	12.987
60	120	0.519	1.3223	-0.3887	13.4638
60	150	0.519	1.2592	-0.3442	14.0363
60	180	0.519	1.3188	-0.3956	14.8159
90	-150	0.4218	1.3369	-0.431	18.4304
90	-120	0.519	1.4531	-0.5324	13.8989
90	-90	0.4192	1.4118	-0.4961	16.8817
90	-60	0.519	1.5046	-0.5827	12.9513
90	-30	0.4198	1.5252	-0.6012	15.7421
90	0	0.42	2.5816	-0.0522	17.4592
90	30	0.519	1.8777	-0.1491	12.4729
90	60	0.519	1.1569	-0.0172	12.8569
90	90	0.519	1.3973	-0.4238	13.2431
90	120	0.519	1.28	-0.3418	13.7262
90	150	0.519	1.3107	-0.3782	14.3465
90	180	0.4205	1.0935	-0.2235	19.8331

Figure 7: Our Control gains by cubic polynomial interpolation algorithm. (a) K_ρ (b) K_α (c) K_β .

The Author's GA table shows the optimal gains, final arrival time, and fitness value computed at all 72 grid points. The arrival times range from approximately 12.02 to 14.68 s.

Our generated table similarly shows the optimal gains, final arrival time, and fitness value computed at all 72 grid points. The arrival times range from approximately 12.0316 to 16.57 s, almost consistent with the author's results.

Although small numerical differences exist when comparing both datasets, the overall gain trends and behavior remain consistent. Below, specific cases are compared in detail.

3.2 Case Studies for Comparison

Case 1:

$$\alpha = -60^\circ, \quad \beta = -150^\circ$$

Author's GA values:

$$K_\rho = 0.5178, \quad K_\alpha = 1.1583, \quad K_\beta = -0.2721, \quad T_f = 13.7590$$

Our GA values:

$$K_\rho = 0.519, \quad K_\alpha = 1.3118, \quad K_\beta = -0.3864, \quad T_f = 14.0702$$

Error:

$$\Delta K_\rho = 0.0012, \quad \Delta K_\alpha = 0.1535, \quad \Delta K_\beta = 0.1143, \quad \Delta T_f = 0.3112$$

Case 60:

$$\alpha = 60^\circ, \quad \beta = 180^\circ$$

Author's GA values:

$$K_\rho = 0.5179, \quad K_\alpha = 1.1138, \quad K_\beta = -0.2571, \quad T_f = 14.37$$

Our GA values:

$$K_\rho = 0.519, \quad K_\alpha = 1.3188, \quad K_\beta = -0.3956, \quad T_f = 14.8159$$

Error:

$$\Delta K_\rho = 0.0012, \quad \Delta K_\alpha = 0.205, \quad \Delta K_\beta = 0.1385, \quad \Delta T_f = 0.4459$$

Observations: There are slight errors in the generated values by GA algorithm; it is not that significant. It is dependent on the upper and lower bound values as well the weights a,b assigned to the fitness functions. The weights used by the author has not been mentioned. We have used a=1 and b=10 in our code.

3.3 Comparison of the cubic polynomial interpolation algorithm

Author's plots:

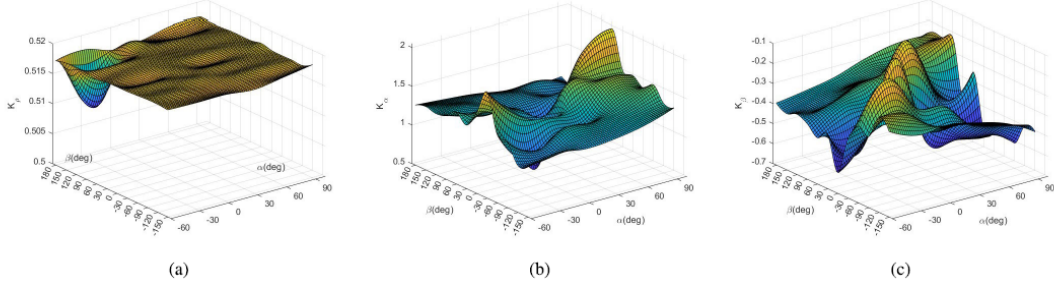


Figure 8: 3D plots of Control gains by cubic polynomial interpolation algorithm by the author. (a) K_ρ (b) K_α (c) K_β .

Our Plots:

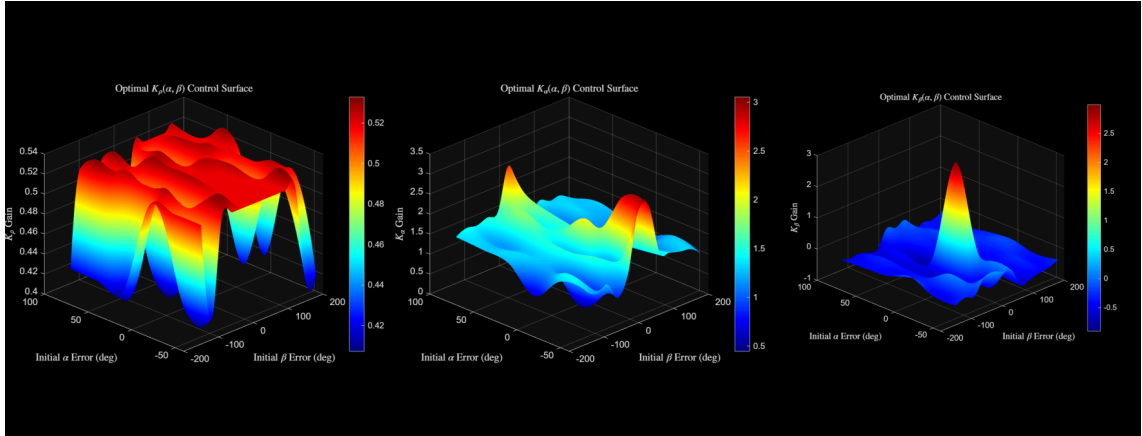


Figure 9: Our 3D plots for Control gains by the cubic polynomial interpolation algorithm. (a) K_ρ (b) K_α (c) K_β .

It appears that overall there is a discrepancy in the generated Optimal gain values, likely caused by different weights used in the Fitness function compared to the author's work. As previously noted, the exact final values for the fitness function were not specified, which could impact the overall system performance. The next section will compare the Pose plots resulting from 12 different initial heading positions.

3.4 Comparison of the multipose plot

Author's plot

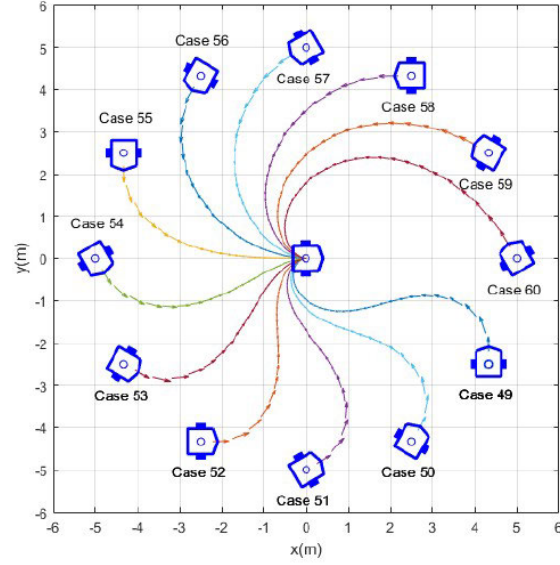


Figure 10: . The optimal path of the mobile robot from the author when the initial conditions are $\alpha(0) = 60^\circ$ and $-180^\circ < \beta(0) < 180^\circ$.

Our Plot

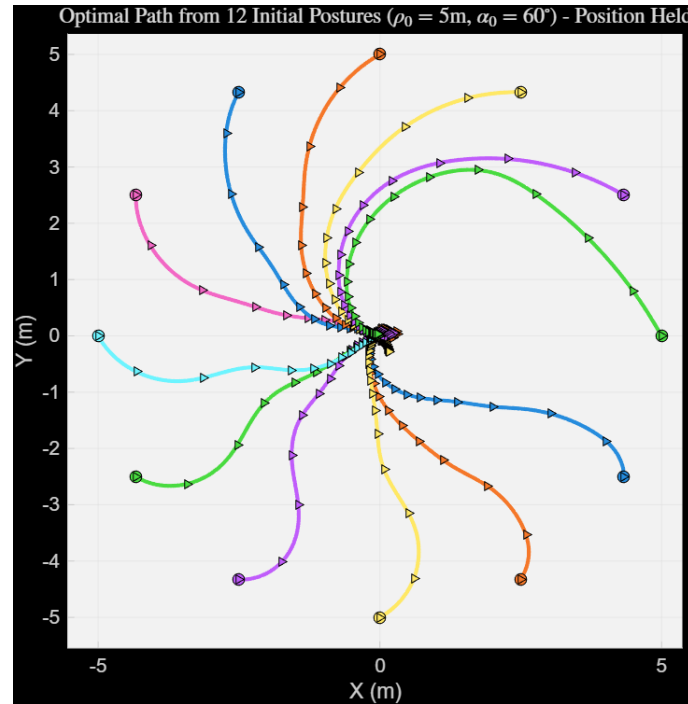


Figure 11: The optimal path of the mobile robot from our results when the initial conditions are $\alpha(0) = 60^\circ$ and $-180^\circ < \beta(0) < 180^\circ$.

The author's pose plots show the mobile robot successfully reaching the goal pose (position (0,0) with near-zero heading angle error). In contrast, our generated plot, while achieving the target position, does not appear to follow an optimal or smooth trajectory.

This sub-optimal path is primarily attributed to the variations in the optimal kinematic gain values ($K_\rho, K_\alpha, K_\beta$) derived from our Genetic Algorithm (GA) implementation compared to the author's. The slight differences in the GA search results (likely due to unknown fitness function weights or settling in different local minima) appear to have significantly impacted the resulting trajectory geometry.

A critical observation in our results is the robot's struggle with the final orientation:

- **Persistent Heading Error:** Our robot is attempting to correct the heading angle error (related to α and β) **after reaching near the goal position**.
- **Failure to Converge:** Despite these efforts, the robot is **unable to converge to the minimal required heading angle error within the defined time interval (T_f)**. This is evident in the final configuration of the error states:

This suggests that the discrepancy in the calculated optimal gains did not just affect the total time to travel (T_f), but fundamentally compromised the kinematic controller's ability to efficiently coordinate linear and angular velocities. This resulted in a poorer final pose convergence and an extended settling time beyond the simulated window, highlighting the sensitivity of the overall system performance to the initial kinematic optimization phase.

3.5 Comparison of 4 Random initial postures

Case	Initial position			K_rho	K_alpha	K_beta	Fitness Tf
	Alpha	Beta					
1	-42	32	Author's values	0.5177	1.4827	-0.3511	12.39
			Our values	0.519	2.9263	0.0751	20.57
2	25	145	Author's values	0.5175	1.1373	-0.2432	14.02
			Our values	0.518	1.114	-0.2538	14.71
3	55	-35	Author's values	0.5179	1.6267	-0.5748	12.6
			Our values	0.5224	1.7464	-0.4138	19.74
4	-5	-125	Author's values	0.5177	1.0889	-0.2334	13.19
			Our values	0.5132	1.1413	-0.2849	17.78

Figure 12: . Comparison of $K_\rho K_\alpha K_\beta$ of 4 random initial postures f.

The plots seen earlier were for defined grid points. When implemented at α and β values other than the grid points, our controller is taking more time to reach the destination for all the heading angles, this maybe due to the dynamic constraint along with the disturbances that the author has considered. The system tries to seek a optimal path with dynamic constraints. The arrival time and optimal path is a tradeoff that we can set using the weight parameters.

4 Conclusions

The author has successfully implemented and analyzed a modular control system for a nonholonomic mobile robot, aiming for optimal trajectory tracking in minimal time while considering dynamic constraints. The approach segmented the problem into two parts: Firstly, the implementation of the kinematic controller using Genetic algorithm to calculate the optimal gain values which helps create a optimal trajectory. Secondly, the implementation of the Adaptive PIF controller using the gradient Descent method where the dynamic of the nonholonomic system was taken into consideration for adaptive tracking. We tried to simulate the same system as similar as possible using the functions defined by the author using all the constraints as same as that mentioned in the journal.

The main difference we noticed was in the initial part itself where we have generated the optimal gain values for 72 grid points. It was implemented in MATLAB using Global Optimization Toolbox to generate the genetic algorithm table. The values generated by author and us showed differences. Also, the author got a consistent final arrival time in generated result. In our case it varied in bigger range. Also, the time required for generating this table is very high. The cubic interpolation of these values was done using the MATLAB function. Author has not mentioned the exact implementation of cubic interpolation.

Thereafter we move into the next section where the main task is to calculate the adaptive PID gains. The discrete equation used by the author has discarded the sampling time due to some reason not mentioned. The equations mentioned by the author has been used for generation of the PID gains. The dynamic equation of the holonomic equation were used to create the final angular velocities of the wheels.

In our implemnetation when the robot is reaching the goal its seems to be loosing the control because of the error pertaining the "forward facing angle" of the robot due to which the robot seems to be moving haphazardly near the goal position. The reason for such behaviour will be studied in the future. One case of this issue seems to be integral windup happening at the end. Maybe the torque disturbances introduced at the goal position is unable to be controlled.

We understand the importance of the dynamics of the system because it relates to the real life challenges. We will be tryig to implement this as a hands on Project to understand the dynamics stablization.

References

- [1] Z. Sun, S. Hu, H. Xie, H. Li, J. Zheng, and B. Chen, “Fuzzy adaptive recursive terminal sliding mode control for an agricultural omnidirectional mobile robot,” *Comput. Electr. Eng.*, vol. 105, Jan. 2023, Art. no. 108529, doi: <https://doi.org/10.1016/j.compeleceng.2022.108529>
- [2] K. Lee and S. Park, “Geometric zone-control algorithm for collision and deadlock avoidance in AGV system,” *IEEE Access*, vol. 11, pp. 131289–131301, 2023 doi: <https://doi.org/10.1109/ACCESS.2023.3335843>
- [3] I. A. Sulistijono, M. Rois, A. Yuniawan, and E. H. Binugroho, “Teleoperated food and medicine courier mobile robot for infected diseases patient,” in *Proc. IEEE Region 10 Conf. (TENCON)*, Dec. 2021, pp. 620–625. doi: <https://doi.org/10.1109/TENCON54134.2021.9707377>
- [4] S.-C. Park, M. Kim, H.-M. Park, and J.-H. Park, “Optimal control gains optimization for mobile robot considering dynamic constraints,” *IEEE Access*, vol. 12, pp. 180079–180092, 2024, doi: <https://doi.org/10.1109/ACCESS.2024.3507739>
- [5] S. J. Al-Kamil and R. Szabolcsi, “Optimizing path planning in mobile robot systems using motion capture technology,” *Results Eng.*, vol. 22, Jun. 2024, Art. no. 102043, doi: <https://doi.org/10.1016/j.rineng.2024.102043>
- [6] J. Płaskonka, “Different kinematic path following controllers for a wheeled mobile robot of (2,0) type,” *J. Intell. Robot. Syst.*, vol. 77, nos. 3–4, pp. 481–498, Mar. 2015, doi: <https://doi.org/10.1007/s10846-013-9879-6>
- [7] D. Wang, J. Zhou, C. He, and Z. Miao, “Experimental verification of formation control in nonholonomic multi-mobile robots,” in *Proc. 40th Chin. Control Conf. (CCC)*, Jul. 2021, pp. 5327–5331, doi: <https://doi.org/10.23919/CCC52363.2021.9550119>
- [8] I. Negrean, C. Schonstein, K. Kacso, C. Negrean, and A. Duca, “Formulations about dynamics of mobile robots,” *Solid State Phenomena*, vols. 166–167, pp. 309–314, Sep. 2010, doi: <https://doi.org/10.4028/www.scientific.net/ssp.166-167.309>
- [9] Y. Choi and J. Park, “Predictive control based on genetic algorithm for mobile robots with constraints,” *J. Korea Inst. Inf. Commun. Eng.*, vol. 22, no. 1, pp. 9–16, Jan. 2018, doi: <https://doi.org/10.6109/jkiice.2018.22.1.9>
- [10] R. Fierro and F. L. Lewis, “Control of a nonholonomic mobile robot using neural networks,” *IEEE Trans. Neural Netw.*, vol. 9, no. 4, pp. 589–600, Jul. 1998, doi: <https://doi.org/10.1109/72.701173>
- [11] S. Nandy, S. N. Shome, G. Chakraborty, and C. S. Kumar, “A modular approach to detailed dynamic formulation and control of wheeled mobile robot,” in *Proc. IEEE Int. Conf. Mechatronics Autom.*, Aug. 2011, pp. 1471–1478, doi: <https://doi.org/10.1109/ICMA.2011.5985967>

- [12] F. N. Martins and A. S. Brandão, “Motion control and velocity-based dynamic compensation for mobile robots,” in *Applications of Mobile Robots*. London, U.K.: IntechOpen, 2019.
- [13] X. Xiao, B. Liu, G. Warnell, and P. Stone, “Motion planning and control for mobile robot navigation using machine learning: A survey,” *Auto. Robots*, vol. 46, no. 5, pp. 569–597, Mar. 2022.
- [14] Y. Cao, K. Ni, T. Kawaguchi, and S. Hashimoto, “Path following for autonomous mobile robots with deep reinforcement learning,” *Sensors*, vol. 24, no. 2, p. 561, Jan. 2024, doi: <https://doi.org/10.3390/s24020561>
- [15] R. Singh, T. K. Bera, and N. Chatti, “A real-time obstacle avoidance and path tracking strategy for a mobile robot using machine-learning and vision-based approach,” *Simulation*, vol. 98, no. 9, pp. 789–805, Apr. 2022, doi: <https://doi.org/10.1177/00375497221091592>
- [16] R. Chai *et al.*, “Design and experimental validation of deep reinforcement learning-based fast trajectory planning and control for mobile robot in unknown environment,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 4, pp. 5778–5792, Apr. 2022.
- [17] H. Liu, Y. Shen, S. Yu, Z. Gao, and T. Wu, “Deep reinforcement learning for mobile robot path planning,” arXiv:2404.06974, 2024, doi: <https://arxiv.org/abs/2404.06974>
- [18] É. Poulin, A. Pomerleau, A. Desbiens, and D. Hodouin, “Development and evaluation of an auto-tuning and adaptive PID controller,” *Automatica*, vol. 32, no. 1, pp. 71–82, Jan. 1996.
- [19] H. Chang and T.-S. Jin, “Adaptive tracking controller based on the PID for mobile robot path tracking,” in *Proc. 6th Int. Conf. Intell. Robot. Appl.*, Busan, South Korea, Jan. 2013, pp. 540–549.
- [20] A. S. Al-Araji, M. F. Abbod, and H. S. Al-Raweshidy, “Design of an adaptive nonlinear PID controller for nonholonomic mobile robot based on posture identifier,” in *Proc. IEEE Int. Conf. Control Syst., Comput. Eng.*, Nov. 2011, pp. 337–342.
- [21] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed. Cambridge, MA, USA: MIT Press, 2011.
- [22] F. L. Lewis, D. Vrabie, and V. L. Syrmos, *Optimal Control*, 3rd ed. Hoboken, NJ, USA: Wiley, 2012.
- [23] M. W. Spong, S. Hutchinson, and M. Vidyasagar, *Robot Modeling and Control*, vol. 3. New York, NY, USA: Wiley, 2006, pp. 75–118.
- [24] H. Choset, K. M. Lynch, S. Hutchinson, G. A. Kantor, and W. Burgard, *Principles of Robot Motion: Theory, Algorithms, and Implementations*. Cambridge, MA, USA: MIT Press, 2005.

- [25] G. Cook and F. Zhang, *Mobile Robots: Navigation, Control and Sensing, Surface Robots and AUVs*. Hoboken, NJ, USA: Wiley, 2020.
- [26] M. Mitchell, *An Introduction to Genetic Algorithms*. Cambridge, MA, USA: MIT Press, 1998.
- [27] J. Holland, “Genetic algorithms,” *Sci. Amer.*, vol. 267, no. 1, pp. 44–50, 1992.
- [28] MATLAB(2025). Accessed: 2025. [Online]. Available: <https://matlab.mathworks.com/>